

Prompt for both AIs:

Generate C++ code using objects, not functions, to implement the following requirements:
The program will prioritize emails for a busy company CEO.

- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap to handle all your email prioritization for the CEO.
- You must create objects from scratch. Do not include pre-existing heap Modules.

- **Test File format:**

Command	Description
EMAIL <sender category>, <subject line>, <date>	The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows: • Boss – read first • Subordinate – read next • Peer – read next • ImportantPerson – read next • OtherPerson – read last If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint. EMAIL is followed by space. The rest of the fields are delimited. Assume <sender category> is one of the five strings listed above. Assume <subject line> is a string which may contain spaces, but not commas Assume <date> is in the format: MM-DD-YYYY
NEXT	Next email for the CEO to read. Display the information on the terminal in the following format: Sender: Subject: Date:
READ	CEO has read email and dealt with it
COUNT	Display current count of unread emails

- The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

- Sample Test File:

```
EMAIL Peer,Can you help me on this?,12-01-2024
EMAIL OtherPerson,Try our product,12-19-2024
EMAIL Boss,Important,12-20-2024
EMAIL Subordinate,How do I handle this?,12-25-2024
EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024
EMAIL Boss,Never Mind,01-03-2025
COUNT
NEXT
READ
NEXT
READ
COUNT
```

- Your program should display this:

There are 6 emails to read.

Next email:

Sender: Boss
Subject: Never Mind
Date: 01-03-2025

Next email:

Sender: Boss
Subject: Important
Date: 12-20-2024

There are 4 emails to read.

- Consider edge cases and build code that handles them.

Analysis:

2. Analyze the two GenAI programs. The analysis must include:

a. Name of two GenAI programs

I used ChatGPT and Claude.

b. How you accessed them (e.g., Google query, CoPilot on my computer)

I accessed both ChatGPT and Claude on my computer web browser.

c. Prompt used. (You must use the same prompt for both GenAIs.)

Prompt used given above - pictures were given as well

d. Generated code from both GenAIs.

Generated code from ChatGPT:

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <string>
#include <vector>

using namespace std;

// -----
// Email object
// -----

class Email {
private:
    string sender;
    string subject;
    string date;
    int arrivalOrder;

public:
    Email() {
        sender = "";
        subject = "";
        date = "";
        arrivalOrder = 0;
    }

    Email(string s, string sub, string d, int order) {
```

```

        sender = s;
        subject = sub;
        date = d;
        arrivalOrder = order;
    }

    string getSender() const {
        return sender;
    }

    string getSubject() const {
        return subject;
    }

    string getDate() const {
        return date;
    }

    int getArrivalOrder() const {
        return arrivalOrder;
    }

    // Gives each sender category a priority number
    int getSenderPriority() const {
        if (sender == "Boss")
            return 5;
        else if (sender == "Subordinate")
            return 4;
        else if (sender == "Peer")
            return 3;
        else if (sender == "ImportantPerson")
            return 2;
        else if (sender == "OtherPerson")
            return 1;

        return 0;
    }

    // Turns MM-DD-YYYY into YYYYMMDD
    // This makes dates easy to compare as integers
    int getDateValue() const {
        int month, day, year;

```

```

        char dash1, dash2;

        stringstream ss(date);

        ss >> month >> dash1 >> day >> dash2 >> year;

        return year * 10000 + month * 100 + day;
    }

    void display() const {
        cout << "Next email:" << endl;
        cout << "\tSender: " << sender << endl;
        cout << "\tSubject: " << subject << endl;
        cout << "\tDate: " << date << endl;
    }
};

// -----
// MaxHeap object
// -----

class MaxHeap {
private:
    vector<Email> heap;

    // Returns true when email a should be placed
    // ahead of email b.
    bool higherPriority(const Email& a, const Email& b) const {

        // First compare sender category
        if (a.getSenderPriority() != b.getSenderPriority()) {
            return a.getSenderPriority() > b.getSenderPriority();
        }

        // Same sender category:
        // newest email should come first
        if (a.getDateValue() != b.getDateValue()) {
            return a.getDateValue() > b.getDateValue();
        }

        // If both sender priority and date are identical,
        // keep the one that arrived first ahead.
    }
};

```

```

        return a.getArrivalOrder() < b.getArrivalOrder();
    }

    void swapEmails(int a, int b) {
        Email temp = heap[a];
        heap[a] = heap[b];
        heap[b] = temp;
    }

    void heapifyUp(int index) {

        while (index > 0) {

            int parent = (index - 1) / 2;

            if (higherPriority(heap[index], heap[parent])) {
                swapEmails(index, parent);
                index = parent;
            }
            else {
                break;
            }
        }
    }

    void heapifyDown(int index) {

        int size = heap.size();

        while (true) {

            int leftChild = 2 * index + 1;
            int rightChild = 2 * index + 2;

            int highest = index;

            if (leftChild < size &&
                higherPriority(heap[leftChild], heap[highest])) {

                highest = leftChild;
            }

```

```

        if (rightChild < size &&
            higherPriority(heap[rightChild], heap[highest])) {

            highest = rightChild;
        }

        if (highest == index) {
            break;
        }

        swapEmails(index, highest);
        index = highest;
    }
}

```

public:

```

    MaxHeap() {
    }

    void insert(const Email& email) {

        heap.push_back(email);

        int newIndex = heap.size() - 1;

        heapifyUp(newIndex);
    }

    bool isEmpty() const {
        return heap.empty();
    }

    int getCount() const {
        return heap.size();
    }

    // NEXT
    void displayNext() const {

        if (isEmpty()) {
            cout << "There are no emails to read." << endl;
            return;
        }
    }

```

```

    }

    heap[0].display();
}

// READ
void removeNext() {

    if (isEmpty()) {
        return;
    }

    // If only one email exists, just remove it
    if (heap.size() == 1) {
        heap.pop_back();
        return;
    }

    // Move the last email to the root
    heap[0] = heap.back();

    // Delete the old last position
    heap.pop_back();

    // Restore MaxHeap order
    heapifyDown(0);
}

// COUNT
void displayCount() const {
    cout << "There are " << heap.size()
         << " emails to read." << endl;
}
};

// -----
// CEO Inbox object
// -----

class CEOInbox {
private:
    MaxHeap emails;

```

```

    int arrivalCounter;

public:
    CEOInbox() {
        arrivalCounter = 0;
    }

    void addEmail(string line) {

        // Remove "EMAIL "
        string information = line.substr(6);

        int firstComma = information.find(',');
        int secondComma = information.find(',', firstComma + 1);

        string sender =
            information.substr(0, firstComma);

        string subject =
            information.substr(
                firstComma + 1,
                secondComma - firstComma - 1
            );

        string date =
            information.substr(secondComma + 1);

        Email newEmail(
            sender,
            subject,
            date,
            arrivalCounter
        );

        arrivalCounter++;

        emails.insert(newEmail);
    }

    void processCommand(string line) {

        if (line.substr(0, 6) == "EMAIL ") {

```

```

        addEmail(line);
    }
    else if (line == "NEXT") {

        emails.displayNext();
    }
    else if (line == "READ") {

        emails.removeNext();
    }
    else if (line == "COUNT") {

        emails.displayCount();
    }
}

void processFile(string fileName) {

    ifstream inputFile(fileName);

    if (!inputFile.is_open()) {
        cout << "Unable to open file." << endl;
        return;
    }

    string line;

    while (getline(inputFile, line)) {

        // Handles Windows-style line endings
        if (!line.empty() && line.back() == '\\r') {
            line.pop_back();
        }

        if (!line.empty()) {
            processCommand(line);
        }
    }

    inputFile.close();
}

```

```
};

// -----
// Main
// -----

int main(int argc, char* argv[]) {

    CEOInbox inbox;

    string fileName;

    // Allows:
    // ./program test.txt
    if (argc > 1) {
        fileName = argv[1];
    }
    else {
        cout << "Enter test file name: ";
        cin >> fileName;
    }

    inbox.processFile(fileName);

    return 0;
}
```

Generated code from Claude:

```
// email_priority_queue.cpp
//
// CEO Email Priority Queue
// -----
// Prioritizes a CEO's inbox using a hand-written, list/array-based MaxHeap
// (no std::priority_queue or <algorithm> heap functions anywhere). Emails
// are ordered first by sender category (Boss > Subordinate > Peer >
// ImportantPerson > OtherPerson) and, within the same category, by date
// (newest first).
//
// Commands (one per line):
//   EMAIL <senderCategory>,<subject>,<date>   -> enqueue a new email
```

```

//  NEXT          -> peek at top email (no removal)
//  READ          -> remove top email (no display)
//  COUNT         -> print number of unread emails
//
// Reads from a file given as a command-line argument, or from stdin if none
// is given, so it works with both "prog file.txt" and "prog < file.txt".

#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <stdexcept>
#include <cctype>

using namespace std;

// -----
// StringUtils - small helper object for whitespace trimming
// -----

class StringUtils {
public:
    static string trim(const string& text) {
        size_t start = 0;
        while (start < text.size() &&
            isspace(static_cast<unsigned char>(text[start]))) {
            start++;
        }
        size_t end = text.size();
        while (end > start &&
            isspace(static_cast<unsigned char>(text[end - 1]))) {
            end--;
        }
        return text.substr(start, end - start);
    }
};

// -----
// Date - represents a MM-DD-YYYY date and knows how to compare itself
// -----

class Date {
public:
    Date() : month(0), day(0), year(0), originalText("") {}

```

```

static Date fromString(const string& text) {
    size_t firstDash = text.find('-');
    size_t secondDash = (firstDash == string::npos)
        ? string::npos
        : text.find('-', firstDash + 1);

    if (firstDash == string::npos || secondDash == string::npos) {
        throw invalid_argument("date '" + text + "' is not in MM-DD-YYYY format");
    }

    Date result;
    result.originalText = text;
    try {
        result.month = stoi(text.substr(0, firstDash));
        result.day = stoi(text.substr(firstDash + 1, secondDash - firstDash - 1));
        result.year = stoi(text.substr(secondDash + 1));
    } catch (const exception&) {
        throw invalid_argument("date '" + text + "' contains non-numeric fields");
    }
    return result;
}

// Returns true if this date comes after (is newer than) other.
bool isNewerThan(const Date& other) const {
    if (year != other.year) return year > other.year;
    if (month != other.month) return month > other.month;
    return day > other.day;
}

string toString() const { return originalText; }

private:
    int month;
    int day;
    int year;
    string originalText;
};

// -----
// SenderCategory - maps a sender category name to a numeric priority
// -----

```

```

class SenderCategory {
public:
    // Higher number = higher priority = read first.
    static int priorityOf(const string& category) {
        if (category == "Boss") return 5;
        if (category == "Subordinate") return 4;
        if (category == "Peer") return 3;
        if (category == "ImportantPerson") return 2;
        if (category == "OtherPerson") return 1;
        throw invalid_argument("unknown sender category '" + category + "'");
    }
};

// -----
// Email - one inbox item; knows how to compare its priority to another
// -----

class Email {
public:
    Email() : priority(0) {}

    Email(const string& senderCategoryIn, const string& subjectIn, const Date& dateIn)
        : senderCategory(senderCategoryIn),
          subject(subjectIn),
          date(dateIn),
          priority(SenderCategory::priorityOf(senderCategoryIn)) {}

    const string& getSenderCategory() const { return senderCategory; }
    const string& getSubject() const { return subject; }
    const Date& getDate() const { return date; }
    int getPriority() const { return priority; }

    // Max-heap ordering: higher sender-category priority wins; ties are
    // broken by the newest date (per the CEO's Sprint-era trick).
    bool hasHigherPriorityThan(const Email& other) const {
        if (priority != other.priority) return priority > other.priority;
        return date.isNewerThan(other.date);
    }

private:
    string senderCategory;
    string subject;
    Date date;

```

```

    int priority;
};

// -----
// MaxHeap - a hand-written, list/array-based (std::vector-backed) binary
// max-heap of Email objects. No pre-existing heap containers/algorithms
// (std::priority_queue, std::make_heap, etc.) are used anywhere here.
// -----

class MaxHeap {
public:
    MaxHeap() {}

    bool isEmpty() const { return items.empty(); }
    size_t size() const { return items.size(); }

    void insert(const Email& email) {
        items.push_back(email);
        siftUp(items.size() - 1);
    }

    // Returns the highest priority email without removing it.
    const Email& peekMax() const {
        if (isEmpty()) {
            throw runtime_error("peekMax() called on an empty heap");
        }
        return items[0];
    }

    // Removes and returns the highest priority email.
    Email extractMax() {
        if (isEmpty()) {
            throw runtime_error("extractMax() called on an empty heap");
        }
        Email maxEmail = items[0];
        items[0] = items.back();
        items.pop_back();
        if (!items.empty()) {
            siftDown(0);
        }
        return maxEmail;
    }
}

```

```

private:
    vector<Email> items; // the "list" backing this list-based heap

    static size_t parentOf(size_t i) { return (i - 1) / 2; }
    static size_t leftChildOf(size_t i) { return 2 * i + 1; }
    static size_t rightChildOf(size_t i) { return 2 * i + 2; }

    void siftUp(size_t index) {
        while (index > 0) {
            size_t parent = parentOf(index);
            if (items[index].hasHigherPriorityThan(items[parent])) {
                swap(items[index], items[parent]);
                index = parent;
            } else {
                break;
            }
        }
    }

    void siftDown(size_t index) {
        size_t n = items.size();
        while (true) {
            size_t left = leftChildOf(index);
            size_t right = rightChildOf(index);
            size_t largest = index;

            if (left < n && items[left].hasHigherPriorityThan(items[largest])) {
                largest = left;
            }
            if (right < n && items[right].hasHigherPriorityThan(items[largest])) {
                largest = right;
            }
            if (largest == index) break;

            swap(items[index], items[largest]);
            index = largest;
        }
    }
};

// -----
// EmailInbox - owns the MaxHeap and implements CEO-facing behavior:

```

```

// adding mail, peeking at the next email, marking one as read, and
// reporting the unread count. This is where command semantics (and the
// edge cases the spec calls out) live.
// -----
class EmailInbox {
public:
    void addEmail(const Email& email) { heap.insert(email); }

    // NEXT: non-destructive peek. Calling this twice in a row with no
    // intervening READ always shows the same email, because nothing is
    // removed here.
    void showNext() const {
        if (heap.isEmpty()) {
            cout << "There are no emails to read." << endl;
            cout << endl;
            return;
        }
        const Email& top = heap.peekMax();
        cout << "Next email:" << endl;
        cout << "\tSender: " << top.getSenderCategory() << endl;
        cout << "\tSubject: " << top.getSubject() << endl;
        cout << "\tDate: " << top.getDate().toString() << endl;
        cout << endl;
    }

    // READ: the CEO dealt with the top email. It is removed with no
    // display. Calling READ twice in a row (no intervening NEXT) removes
    // two different emails, neither one ever shown - exactly as specified.
    // If the inbox is empty, this is a safe no-op.
    void markTopAsRead() {
        if (heap.isEmpty()) {
            return;
        }
        heap.extractMax();
    }

    void showCount() const {
        cout << "There are " << heap.size() << " emails to read." << endl;
        cout << endl;
    }

private:

```

```

    MaxHeap heap;
};

// -----
// EmailPriorityApp - reads command lines from an input stream and
// dispatches each one to the EmailInbox. Handles blank lines and
// malformed input defensively instead of crashing.
// -----

class EmailPriorityApp {
public:
    void run(istream& input) {
        string rawLine;
        while (getline(input, rawLine)) {
            handleLine(rawLine);
        }
    }

private:
    EmailInbox inbox;

    void handleLine(const string& rawLine) {
        string line = StringUtils::trim(rawLine);
        if (line.empty()) return; // ignore blank lines

        size_t spacePos = line.find(' ');
        string command = (spacePos == string::npos) ? line : line.substr(0, spacePos);

        if (command == "EMAIL") {
            handleEmailCommand(line, rawLine);
        } else if (command == "NEXT") {
            inbox.showNext();
        } else if (command == "READ") {
            inbox.markTopAsRead();
        } else if (command == "COUNT") {
            inbox.showCount();
        } else {
            cerr << "Warning: ignoring unrecognized command: " << rawLine << endl;
        }
    }

    void handleEmailCommand(const string& line, const string& rawLine) {
        // line looks like: EMAIL <senderCategory>,<subject>,<date>
    }
};

```

```

        string rest = StringUtilsils::trim(line.substr(5));

        size_t firstComma = rest.find(',');
        size_t lastComma = rest.rfind(',');

        if (firstComma == string::npos || lastComma == string::npos ||
            firstComma == lastComma) {
            cerr << "Warning: malformed EMAIL command, skipping: " << rawLine << endl;
            return;
        }

        string senderCategory = StringUtilsils::trim(rest.substr(0, firstComma));
        string subject = StringUtilsils::trim(
            rest.substr(firstComma + 1, lastComma - firstComma - 1));
        string dateText = StringUtilsils::trim(rest.substr(lastComma + 1));

        try {
            Date date = Date::fromString(dateText);
            Email email(senderCategory, subject, date);
            inbox.addEmail(email);
        } catch (const exception& e) {
            cerr << "Warning: could not add email (" << e.what() << "): "
                << rawLine << endl;
        }
    }
};

// -----
// main - reads from a file named on the command line if given, otherwise
// from standard input.
// -----
int main(int argc, char* argv[]) {
    EmailPriorityApp app;

    if (argc > 1) {
        ifstream file(argv[1]);
        if (!file.is_open()) {
            cerr << "Error: could not open file '" << argv[1] << "'" << endl;
            return 1;
        }
        app.run(file);
    } else {

```

```
        app.run(cin);  
    }  
  
    return 0;  
}
```

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.

Correctness: In general, both the Claude and ChatGPT code was correct. They both implement MaxHeap from scratch and prioritize email by sender category and then by date. They also each implement NEXT, COUNT, and READ. ChatGPT, however, includes an arrivalOrder feature that orders emails by arrival order if sender category and date are the same. Claude does not do this, which is a slight downside, because behavior becomes unpredictable. However, Claude is stronger when it comes to input handling. ChatGPT assumes commas and dates are formatted correctly and does not trim whitespace. Claude is more proactive. It trims whitespace, checks for misformatted input, and ensures the sender category is from the given list.

Execution time: In terms of BigO, the two programs perform identically. The EMAIL function is $O(\log n)$, because it moves upward through MaxHeap. NEXT is $O(1)$. READ is $O(\log n)$, and COUNT is $O(1)$. However, there is a difference between ChatGPT and Claude when it comes to email comparisons. ChatGPT continuously calls functions to get sender priorities and date values. Each call to get a date value also requires converting the date string to a numeric value. Claude creates email and date objects that make comparison processes quicker. This is because the value processing only has to happen one time, instead of repeatedly. Because of these differences, the Claude program likely runs slightly faster than the ChatGPT program.

Maintainability: Both ChatGPT and Claude use descriptive class names and functions. Both ChatGPT and Claude do not provide sufficient comments. Claude describes a few functions and gives prologue comments that describe the general functionality of the program. However, there are still gaps and a lack of thorough function descriptions. ChatGPT's comments are even more limited than Claude's. Further, Claude separates responsibilities into classes, making tasks more distinguishable and maintainable. It has classes for handling whitespace, date comparison, representing emails, handling the heap, and email

prioritization. The ChatGPT code combines these responsibilities into a few classes, namely a CEOInbox class and an Email class. This overall format makes the Claude code more maintainable because it has more comments and distinct function classes.

Space Complexity: Both the ChatGPT code and the Claude code have a space complexity of $O(n)$ because the space used grows with the number of emails that get stored. There are some differences within the storage methods used for emails, though. ChatGPT simply stores the email, sender, subject, date, and arrival order as strings and integers. Claude, however, stores the date as a Date object. Further, Claude stores sender priority within Email. This means that Claude stores slightly more processed information, which takes up a bit more space. Overall, ChatGPT is slightly more space efficient. However, the difference is minor.

f. Select one to use as the basis for this assignment and justify why you picked it.

I picked Claude Code to improve upon. I picked this one because it is stronger than the ChatGPT code in terms of Maintainability, execution time, and correctness. Claude is slightly less predictable than ChatGPT in terms of ties. However, it is far better at handling bad and varying inputs. Further, its commenting is a bit stronger than ChatGPT's. Further, because of its use of objects to store priority, Claude's execution time is better than ChatGPT's. Space complexity differences are minor. Overall, Claude is a better foundation to start from.

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

Correctness: To improve the correctness of the Claude Code, I added a tiebreaker method so that if the sender category and date are the same, arrival time is used as a tiebreaker. Additionally, the program now validates the date ranges to ensure the given date is possible.

Execution time: To improve execution time, I changed the code so that it does not make a copy of Email in ExtractMax. This prevents the program from needing to go through the time it takes to copy the object.

Maintainability: To improve the maintainability of the code, I added comments to every line. I also added thorough function and class descriptions. This will make editing and

general understanding more efficient for future changes. Further, I changed variable names to be more descriptive. For example, I changed “items” to “emails” and I changed “n” to “heapSize”.

Space Complexity: There were no meaningful improvements that could be made to the Claude code without impacting execution. One change could be to store fewer variables and calculate priority more often. However, doing this would undo improvements made to execution time for little gain. Because of this, I did not make any changes to space complexity.